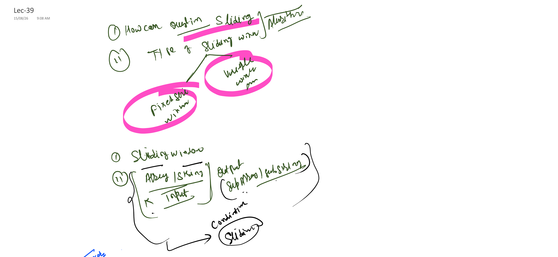

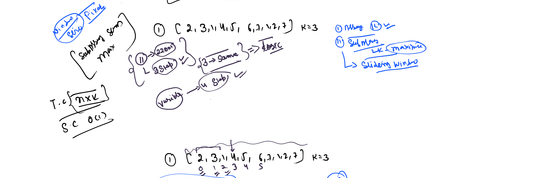

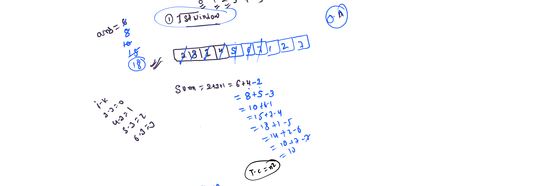

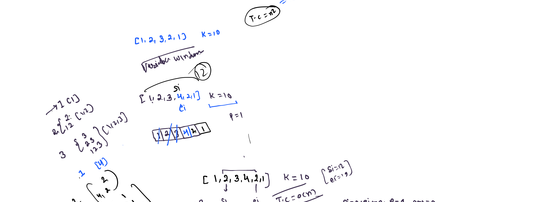

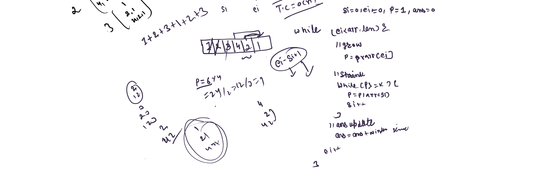

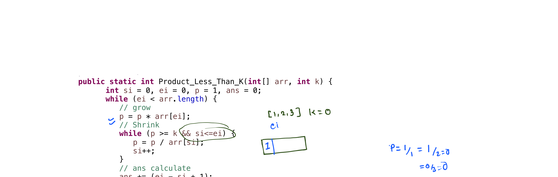

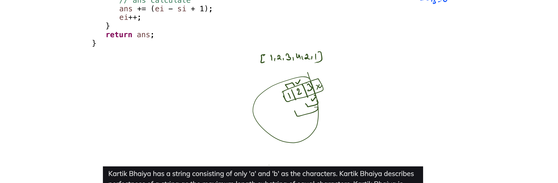

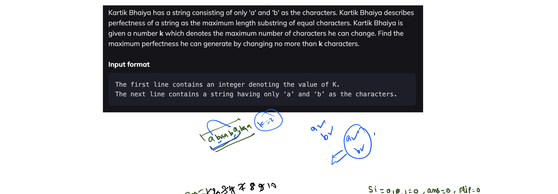

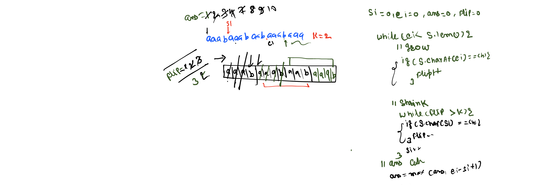

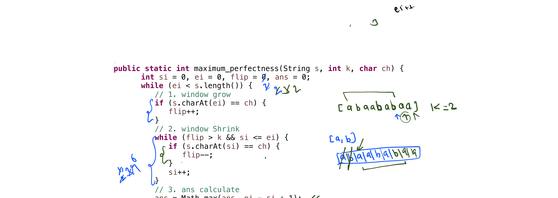

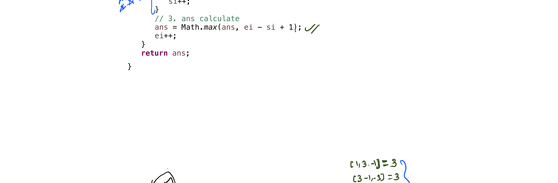

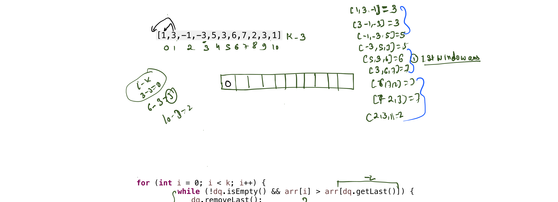

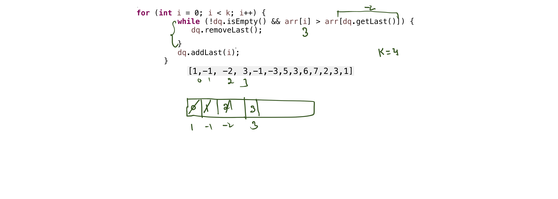

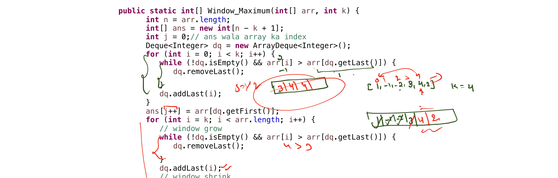

```
public static int Product_Less_Than_K(int[] arr, int k) {
    int si = 0, ei = 0, p = 1, ans = 0;
    while (ei < arr.length) {
        // grow
        p = p * arr[ei];
        // Shrink
        while (p >= k && si<=ei) {
            p = p / arr[si];
            si++;
        }
        // ans calculate
        ans += (ei - si + 1);
        ei++;
    }
    return ans;
}
```


Kartik Bhaiya has a string consisting of only 'a' and 'b' as the characters. Kartik Bhaiya describes perfectness of a string as the maximum length substring of equal characters. Kartik Bhaiya is given a number **k** which denotes the maximum number of characters he can change. Find the maximum perfectness he can generate by changing no more than **k** characters.

Input format

The first line contains an integer denoting the value of K.
The next line contains a string having only 'a' and 'b' as the characters.


```
public static int maximum_perfectness(String s, int k, char ch) {
    int si = 0, ei = 0, flip = 0, ans = 0;
    while (ei < s.length()) {
        // 1. window grow
        if (s.charAt(ei) == ch) {
            flip++;
        }
        // 2. window Shrink
        while (flip > k && si <= ei) {
            if (s.charAt(si) == ch) {
                flip--;
            }
            si++;
        }
        // 3. ans calculate
        ans = Math.max(ans, ei - si + 1);
        ei++;
    }
    return ans;
}
```



```
for (int i = 0; i < k; i++) {
        while (!dq.isEmpty() && arr[i] > arr[dq.getLast()]) {
            dq.removeLast();
        }
        dq.addLast(i);
    }
```



```
public static int[] Window_Maximum(int[] arr, int k) {
    int n = arr.length;
    int[] ans = new int[n - k + 1];
    int j = 0;// ans wala array ka index
    Deque<Integer> dq = new ArrayDeque<Integer>();
    for (int i = 0; i < k; i++) {
        while (!dq.isEmpty() && arr[i] > arr[dq.getLast()]) {
            dq.removeLast();
        }
        dq.addLast(i);
    }
    ans[j++] = arr[dq.getFirst()];
    for (int i = k; i < arr.length; i++) {
        // window grow
        while (!dq.isEmpty() && arr[i] > arr[dq.getLast()]) {
            dq.removeLast();
        }
        dq.addLast(i);
        // window shrink
        if(i-k==dq.getFirst()) {
            dq.removeFirst();
        }
        // ans update
        ans[j++] = arr[dq.getFirst()];
    }
    return ans;
}
```